

Justificación de Decisiones de Diseño – Funcionalidad de Inscripción a Actividades

El presente documento tiene como objetivo justificar las decisiones de diseño adoptadas en la implementación de la funcionalidad de inscripción a actividades dentro de la aplicación. La función principal ``inscribirse_a_actividad()`` y su correspondiente conjunto de pruebas automatizadas en ``test_inscripcion.py`` fueron desarrolladas bajo un enfoque de **calidad**, **claridad** y **mantenibilidad** del código, garantizando el cumplimiento de los criterios de aceptación definidos en la User Story correspondiente.

1. Estructura y modularidad del código

Se decidió ubicar la función ``inscribirse_a_actividad()`` dentro del módulo ``app/inscripcion.py``, manteniendo así una estructura modular que separa la lógica de negocio del código de prueba. Esta decisión facilita la escalabilidad del proyecto y promueve las buenas prácticas de organización en proyectos de Python.

2. Validaciones explícitas y controladas mediante excepciones

- Verificación de aceptación de términos y condiciones.
- Control de disponibilidad de cupos para la actividad seleccionada.
- Validación del talle de vestimenta en caso de ser requerido.
- Comprobación de que el horario seleccionado se encuentra disponible.

Cada validación, en caso de no cumplirse, genera una excepción con un mensaje descriptivo. Esto permite detectar de forma inmediata la causa del error y facilita la escritura de pruebas unitarias que verifiquen estos comportamientos específicos.

3. Uso de pruebas automatizadas con Pytest

Se adoptó el framework **Pytest** por su simplicidad, legibilidad y compatibilidad con entornos de integración continua. Todas las pruebas relacionadas con esta User Story se agrupan en un único archivo (``test_inscripcion.py``), siguiendo una convención de trazabilidad directa entre los tests y la funcionalidad que verifican.

3.1. Enfoque de Desarrollo Conducido por Pruebas (TDD)

La funcionalidad fue implementada utilizando el enfoque **Test-Driven Development (TDD)**, siguiendo el ciclo **Red–Green–Refactor**:

- **Red:** Se definieron inicialmente las pruebas automatizadas que representaban los criterios de aceptación de la User Story, asegurando que fallaran al no existir aún la funcionalidad.
- **Green:** Se implementó el código mínimo necesario en ``inscribirse_a_actividad()`` para que las pruebas pasaran exitosamente.

- Refactor: Finalmente, se revisó y optimizó el código, manteniendo la claridad y eliminando redundancias sin alterar el comportamiento probado.

Este proceso permitió construir una solución fiable, orientada a la calidad y con una trazabilidad directa entre las pruebas y el comportamiento esperado del sistema.

4. Cobertura de casos de prueba

- Inscripción exitosa con todos los datos válidos.
- Intento de inscripción sin cupos disponibles.
- Inscripción sin requerir talle en actividades que no lo necesitan.
- Intento de inscripción en horario no disponible.
- Intento de inscripción sin aceptar términos y condiciones.
- Falta de talle en actividad que lo requiere.

Esta cobertura integral asegura que la función se comporte correctamente en todo el rango de situaciones posibles.

5. Elección del manejo de datos mediante estructuras simples (diccionarios)

Se optó por utilizar estructuras de datos básicas como diccionarios para representar tanto las actividades como los visitantes. Esta decisión facilita la comprensión del código y permite realizar modificaciones rápidas durante las pruebas iniciales del sistema, sin necesidad de implementar clases o modelos complejos, manteniendo la simplicidad necesaria para el alcance actual de la User Story.

6. Enfoque en claridad y mantenibilidad

El código fue diseñado siguiendo principios de **claridad**, **legibilidad** y **responsabilidad única**. Cada sección cumple una función bien definida, evitando la duplicación de lógica. Además, los mensajes de error fueron cuidadosamente redactados para que sean comprensibles por el usuario o por los desarrolladores que realicen mantenimiento futuro.

7. Uso de entorno virtual (virtualenv)

La decisión de ejecutar las pruebas dentro de un entorno virtual garantiza el aislamiento de dependencias y evita conflictos con otras instalaciones del sistema. Esta práctica es estándar en entornos profesionales de desarrollo y asegura la reproducibilidad de los resultados.

Conclusión

Las decisiones de diseño tomadas priorizan la confiabilidad y escalabilidad del sistema. La estructura modular, las validaciones detalladas y la cobertura de pruebas exhaustiva aseguran un desarrollo controlado, mantenible y alineado con las buenas prácticas del desarrollo ágil.